

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

Naskah Praktikum 1

Database Performance Tuning

Jumat, 18 September 2026

Oleh:

13523XXX - Nama 1

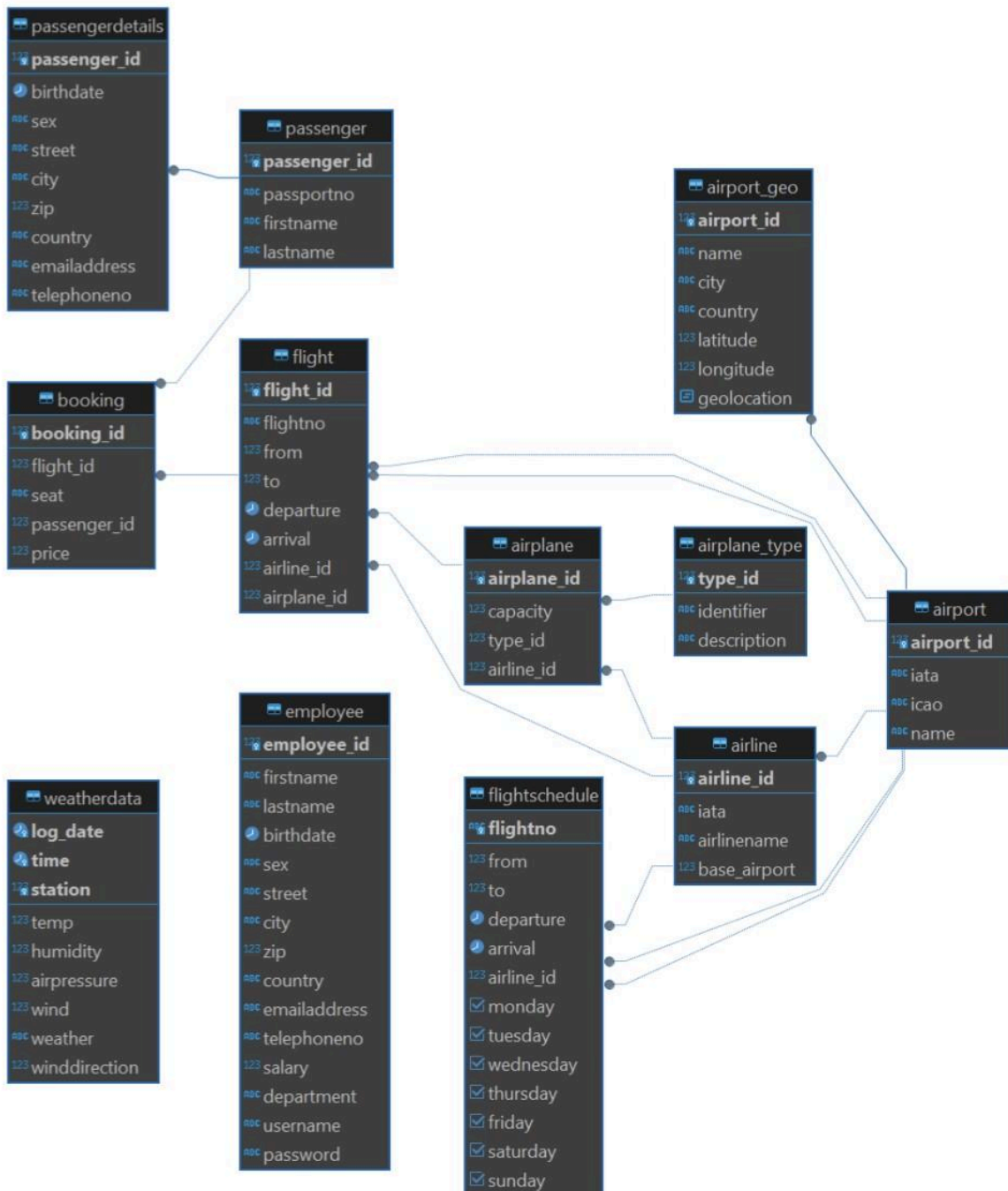
13523XXX - Nama 2

IF3140 - Sistem Basis Data
2026

Petunjuk

1. Kerjakan setiap soal praktikum ini dengan baik
2. Query pengujian pada praktikum ini adalah query dengan makna semantik yang sama dengan query awal. Silakan dibuat sendiri jika tidak diberikan
3. Nama database: airport
4. **Dilarang Menyontek**
5. **docker exec praktikum-db-1 pg_dump -U postgres -d airport -h localhost -p 5432 > [DATABASE_FILE_EXPORTED].sql**

AirportDB



1. Performance Tuning Theory

Jawab pertanyaan berikut ini dengan baik. Seluruh *query* tidak perlu dijalankan pada basis data karena data tidak tersedia. Namun, setiap *query* yang dipertanyakan harus ditulis secara **SINGKAT**, **LENGKAP**, dan **TEPAT**. Jawaban panjang tidak akan menambah nilai.

- a. Dalam sistem AirportDB, admin sering menampilkan daftar booking yang berisi booking_id, flightno, passenger firstname, passenger lastname, seat, dan price. Data booking terus bertambah sehingga query tersebut semakin sering melakukan join antara tabel booking, passenger, dan flight. Apa metode performance tuning yang paling tepat untuk kasus tersebut dan trade-off apa yang ditimbulkan?

- b. Laporan pendapatan penerbangan sering digunakan untuk menampilkan total price booking per airline dan per bulan. Jumlah data pada tabel booking terus meningkat sehingga proses agregasi dan join membutuhkan waktu semakin lama. Apa metode performance tuning yang paling tepat untuk meningkatkan performa laporan tersebut dan trade-off apa yang ditimbulkan?

- c. Database AirportDB mendapatkan penambahan data yang sangat signifikan. Apabila query berikut sering dijalankan setiap hari, metode performance tuning apa yang paling tepat digunakan? Sertakan atribut yang terlibat dalam implementasi performance tuning. Alasan tidak perlu dituliskan.

Frequent Query	Jawaban
SELECT * FROM passenger WHERE passportno = {passport};	
SELECT * FROM airport WHERE iata = {iata_code};	
SELECT * FROM booking WHERE passenger_id = {passenger_id} AND flight_id = {flight_id};	

- d. Tabel passenger menyimpan informasi identitas penumpang berupa passportno, firstname, dan lastname, sedangkan informasi detail seperti street, city, zip, country, emailaddress, dan telephonenumber juga

disimpan pada tabel yang sama. Sistem memiliki fitur pencarian/login penumpang yang hanya membutuhkan `passenger_id`, `passportno`, `firstname`, dan `lastname`. Apa metode performance tuning yang paling tepat untuk kasus tersebut dan trade-off apa yang ditimbulkan?

2. Pakai Apa? Pakai Nanya!

Bibi Tri telah mengumpulkan tiga pertanyaan SQL yang paling sering dijalankan di basis data miliknya. Beliau ingin kamu mengoptimasi ketiga kueri dengan memakai teknik *indexing* yang sesuai.

Untuk tiap pertanyaan, terapkan teknik *indexing* yang sesuai (B-Tree/Hash). Tulis “Indexing tidak cocok” jika *query* tidak cocok dioptimasi dengan *indexing*. Tampilkan keluaran EXPLAIN ANALYZE sebelum dan sesudah optimasi dan beri penjelasan justifikasinya.

```
-- Query 1
SELECT * FROM airport WHERE name LIKE '%Intl%';

-- Query 2
SELECT * FROM employee WHERE country = 'Poland';

-- Query 3
SELECT flight_id, flightno, departure FROM flight
WHERE "from" = 101 ORDER BY departure ASC LIMIT 5;
```

Query 1	SELECT * FROM airport WHERE name LIKE '%Intl%';
Solusi Indexing SQL	[SQL Command/"Index tidak cocok"]
SS Hasil EXPLAIN ANALYZE Sebelum Manipulasi	
SS Hasil EXPLAIN ANALYZE Setelah Manipulasi	
Justifikasi	Tulis dalam poin: 1. Analisis Kebutuhan Kueri 2. Justifikasi Index (jelaskan mekanismenya secara singkat) 3. Justifikasi Kolom (jika dan hanya jika pakai index) 4. Analisis EXPLAIN ANALYZE (jika dan hanya jika pakai index)

Query 2	SELECT * FROM employee WHERE country = 'Poland';
---------	--

Solusi Indexing SQL	[SQL Command/"Index tidak cocok"]
SS Hasil EXPLAIN ANALYZE Sebelum Manipulasi	
SS Hasil EXPLAIN ANALYZE Setelah Manipulasi	
Justifikasi	Tulis dalam poin: 1. Analisis Kebutuhan Kueri 2. Justifikasi Index (jelaskan mekanismenya secara singkat) 3. Justifikasi Kolom (jika dan hanya jika pakai index) 4. Analisis EXPLAIN ANALYZE (jika dan hanya jika pakai index)

Query 3	<pre>SELECT flight_id, flightno, departure FROM flight WHERE "from" = 101 ORDER BY departure ASC LIMIT 5;</pre>
Solusi Indexing SQL	[SQL Command/"Index tidak cocok"]
SS Hasil EXPLAIN ANALYZE Sebelum Manipulasi	
SS Hasil EXPLAIN ANALYZE Setelah Manipulasi	
Justifikasi	Tulis dalam poin: 1. Analisis Kebutuhan Kueri 2. Justifikasi Index (jelaskan mekanismenya secara singkat) 3. Justifikasi Kolom (jika dan hanya jika pakai index) 4. Analisis EXPLAIN ANALYZE (jika dan hanya jika pakai index)

3. Duh, Anak Magang!

Terbang Terus Pantang Mundur Airlines, maskapai bertarif murah dengan slogan "Murah, Asal Sabar", menyimpan semua data penumpangnya dalam satu tabel bernama **passengerdetails**. Tabel ini dibuat oleh seorang anak magang yang kurang mementingkan efisiensi.

Tim analis data rutin diminta membuat laporan tren demografi penumpang. Masalahnya, query sederhana untuk mengambil data penumpang inti pun berjalan sangat lambat. Padahal untuk laporan awal, yang benar-benar mereka butuhkan hanya **passenger_id**, **birthdate**, **sex**, dan **country**. Kolom **street**, **city**, **zip**, **emailaddress**, dan **telephonenumber** nyaris tidak pernah disentuh.

Pak Henzy, manajer tim DBA menugaskan kalian (para praktikan) untuk melakukan performance tuning **TANPA MENGUBAH TABEL AWAL** (membuat tabel baru diperbolehkan).

Gunakan perintah explain dan timing untuk mendapatkan variabel analisis! EXPLAIN digunakan untuk melakukan analisis perbandingan cost sebelum dan sesudah melakukan performance tuning. Jika data terlalu panjang, gunakan \q untuk melihat hasil dari timing.

Pada bagian ini, buatlah query untuk mendapatkan data yang sering diperlukan tim analis!

Query	
SS Hasil Query	

Penjelasan Permasalahan dan Jenis <i>Schema Tuning</i> yang dilakukan	
Saran Perbaikan dan Justifikasinya	
Query untuk Melakukan Perbaikan	

SS Query Perbaikan	
Query Testing Setelah Perbaikan	
SS Query Testing setelah Perbaikan	
Kesimpulan Akhir dan Alasan	

4. SkyRevenue

Sebuah perusahaan pengelola bandara menggunakan basis data Airport untuk menampilkan total nilai pemesanan tiket berdasarkan negara bandara keberangkatan pada periode Februari–Mei 2025. Laporan tersebut juga menampilkan rata-rata total nilai pemesanan antarnegara.

Laporan diakses berulang kali oleh banyak pengguna melalui dashboard analisis. Setiap kali dashboard dibuka, sistem menjalankan query berikut:

Query	<pre> SELECT ag.country, SUM(b.price) AS total_revenue, AVG(SUM(b.price)) OVER () AS avg_revenue FROM booking b INNER JOIN flight f ON b.flight_id = f.flight_id INNER JOIN airport a ON f."from" = a.airport_id INNER JOIN airport_geo ag ON a.airport_id = ag.airport_id WHERE EXTRACT(YEAR FROM f.departure) = 2025 AND EXTRACT(MONTH FROM f.departure) BETWEEN 2 AND 5 GROUP BY ag.country ORDER BY SUM(b.price) DESC; </pre>
-------	---

Tugas anda sebagai seorang ahli basis data dengan kemampuan +1000 social credit  adalah:

1. Menganalisis permasalahan jika query tersebut dijalankan berulang kali oleh banyak pengguna.
2. Memberikan solusi untuk mengatasi permasalahan tersebut beserta query implementasi dan pengujiannya.

Asumsikan sekarang adalah tahun 2026 dan data untuk periode tersebut sudah final serta tidak akan berubah lagi. Hasil laporan tidak boleh berubah, tetapi query pembacaan boleh diganti untuk menggunakan hasil yang telah disimpan.

SS Hasil Query	
Penjelasan Permasalahan	
Saran Perbaikan dan Justifikasinya	
Query untuk Melakukan Perbaikan	
SS Query Perbaikan	
Query Pengujian	
SS Query Masukan dan Hasil setelah Perbaikan	
Kesimpulan Akhir dan Alasan	

5. Wong Tidak Teliti

Mr Boss mengadakan sebuah lomba penerbangan terbanyak oleh seorang penumpang. Pemenangnya akan dihadiahkan tanda tangan Messi 🐐, sehingga jutaan orang sangat mengantisipasi kompetisi ini.

Wong, seorang intern, diberi tanggung jawab untuk membantu tim marketing menyiapkan laporan leaderboard yang akan diperbarui dan dipublikasikan setiap hari (misalnya untuk diposting di media sosial dan dikirim ke tim promosi). Wong dan teman-temannya Cloude 🟠, Chat PPT 🟢, dan Geminini 🔵 berhasil membuat query-nya dengan cepat, lalu langsung dipakai tanpa banyak dipikirkan lebih jauh.

Awalnya semua baik-baik saja. Namun setelah kompetisi berjalan beberapa minggu dan jumlah booking membesar hingga jutaan baris, tim marketing mulai komplain, setiap kali mereka menjalankan laporan harian ini, prosesnya butuh waktu sangat lama bahkan sering membuat mereka terlambat mempublikasikan update leaderboard yang dijanjikan setiap pagi. Berikut query yang mereka gunakan:

```
SELECT
  p.passenger_id,
  CONCAT(p.firstname, ' ', p.lastname) AS fullname,
  COUNT(b.booking_id)      AS total_flights,
  SUM(b.price)             AS total_spend,
  MIN(f.departure)         AS first_flight,
  MAX(f.departure)         AS last_flight
FROM passenger p
JOIN booking b   ON b.passenger_id = p.passenger_id
JOIN flight f    ON f.flight_id = b.flight_id
GROUP BY p.passenger_id, fullname
ORDER BY total_spend DESC;
```

Mr Boss 😡 pun memarahi Wong dan teman-temannya, karena keterlambatan ini membuat tim marketing kesulitan menjaga jadwal publikasi harian. Mereka diminta mencari solusi agar laporan ini bisa dijalankan dengan jauh lebih cepat setiap harinya.

Tugas Anda:

Jelaskan mengapa eksekusi query di atas bisa berjalan lambat.

Berikan solusi menggunakan denormalisasi agar laporan leaderboard harian ini bisa diakses jauh lebih cepat oleh tim marketing.

Catatan Penting: Pastikan solusi yang diberikan tidak mengubah/menghapus data asli pada tabel booking, flight, atau passenger.

Query Masukan	[Query yang ditampilkan di atas tersebut]
SS Hasil Query Masukan	

Penjelasan Permasalahan	
Saran Perbaikan dan Justifikasinya	
Query untuk Melakukan Perbaikan	

6. Pilih Pesawat yang Mana

Yeho sedang membantu sebuah maskapai menentukan pesawat yang cocok untuk berbagai penerbangan. Masalahnya, Yeho ternyata cukup sering salah membedakan pesawat kecil dan pesawat besar. Setiap melihat data pesawat, ia harus scroll melewati banyak sekali baris hanya untuk mencari pesawat dengan kapasitas yang sesuai.

Karena mulai lelah scrolling dan takut suatu hari memilih pesawat berkapasitas 100 penumpang untuk penerbangan yang membutuhkan 300 kursi, Yeho ingin mengatur data pesawat menjadi dua kelompok berdasarkan kapasitasnya.

Bantulah Yeho mengusulkan solusi yang sesuai agar data pesawat dengan kapasitas kurang dari 200 penumpang dapat dikelola secara terpisah dari pesawat dengan kapasitas 200 penumpang atau lebih. Tuliskan query yang diperlukan untuk menerapkan solusi tersebut.

Note: Pastikan agar apapun itu strategi yang dilakukan, tabel-tabel awal pada basis data tidak hilang.

Saran Perbaikan dan Justifikasi	
Query	
SS Hasil Query	

Pembagian Kerja

NIM	Pembagian Kerja
13523000	Makan
13523999	Tidur